2026

Intelligent RAG-Driven Customer Support Ecosystem - LLD

DATA SCIENCE INTERNSHIP – FEBRUARY 2026

VEDIKA PUNJABI - IN226101802

Table of Contents

1. Module-Level Design 2
2. Data Structures & Schema Design 3
3. Workflow Design (LangGraph Implementation) 3
   3.1 Node Definitions 3
   3.2 State Management 3
4. Conditional Routing Logic 4
   4.1 Routing Decision Gate 4
   4.2 Impact on System Reliability 4
5. HITL Design & Integration 5
6. API / Interface Design 5
7. Error Handling 5

1. Module-Level Design

This section outlines the internal logic of the software modules required to execute the RAG system.

- **Document Processing Module:** Uses `PyPDFLoader` to parse the Arkansas Extension PDF into a list of LangChain `Document` objects containing page content and metadata.
- **Chunking Module:** Implements `RecursiveCharacterTextSplitter`. Logic: Splits text by `\n\n`, then `\n`, then space, maintaining a 1000-character size with a 150-character overlap for context preservation.
- **Embedding Module:** Utilizes `gemini-embedding-001` to generate 768-dimensional vector representations.
- **Vector Storage Module:** Employs `ChromaDB` for local persistent storage, allowing for rapid nearest-neighbour vector lookups.
- **Retrieval Module:** Configured as a vector store retriever using Similarity Search($k = 3$) to provide grounded context to the LLM.
- **Query Processing Module:** Uses **Gemini 2.5 Flash Lite** with a system prompt to evaluate if the retrieved context is sufficient for an answer.
- **Graph Execution Module:** Managed by **LangGraph**, handling the orchestration of nodes and state maintenance.
- **HITL Module:** Uses the LangGraph `interrupt` and `checkpoint` mechanism to pause the graph for human intervention.

2. Data Structures & Schema Design

While the HLD defined the system components, the LLD defines the exact structure of the data as it exists in memory and storage.

- **Document representation:** LangChain `Document` class (Content: String, Metadata: Dict).
- **Chunk format:** String segments of ~1000 characters.
- **Embedding structure:** Float array (Vector) of size 768.
- **Query-response schema:** JSON-like structure containing the user's string input and the model's generated string response.
- **State object for graph:** A `TypedDict` containing:
  - ✓ `messages`: List of base messages (History).
  - ✓ `needs_escalation`: Boolean flag.

3. Workflow Design (LangGraph Implementation)

The system's execution is managed through a stateful graph that coordinates the lifecycle of a user request. This design ensures that the conversation context and decision-making logic are preserved at every step.

3.1 Node Definitions

- **START Node:** The entry point where the user's query is initialized into the graph state.
- **ASSISTANT Node:** The primary processing unit. It executes two parallel tasks:
  - ✓ **Context Retrieval:** Queries the ChromaDB vector store for the top 3 most relevant policy chunks.
  - ✓ **Generative Analysis:** Uses Gemini 2.5 Flash Lite to synthesize an answer based strictly on the retrieved chunks.
- **HUMAN REVIEW Node:** A specialized intervention node. When triggered, it executes an `interrupt()`, which halts graph execution and saves the current state to the persistent memory checkpointer, awaiting external human input.

3.2 State Management

The workflow utilizes a **Persistent Checkpointer (`MemorySaver`)**. This allows the graph to "sleep" during human review without losing the session's data, ensuring that when the human provides a response via `Command(resume=...)`, the graph can successfully transition to the **END** state.

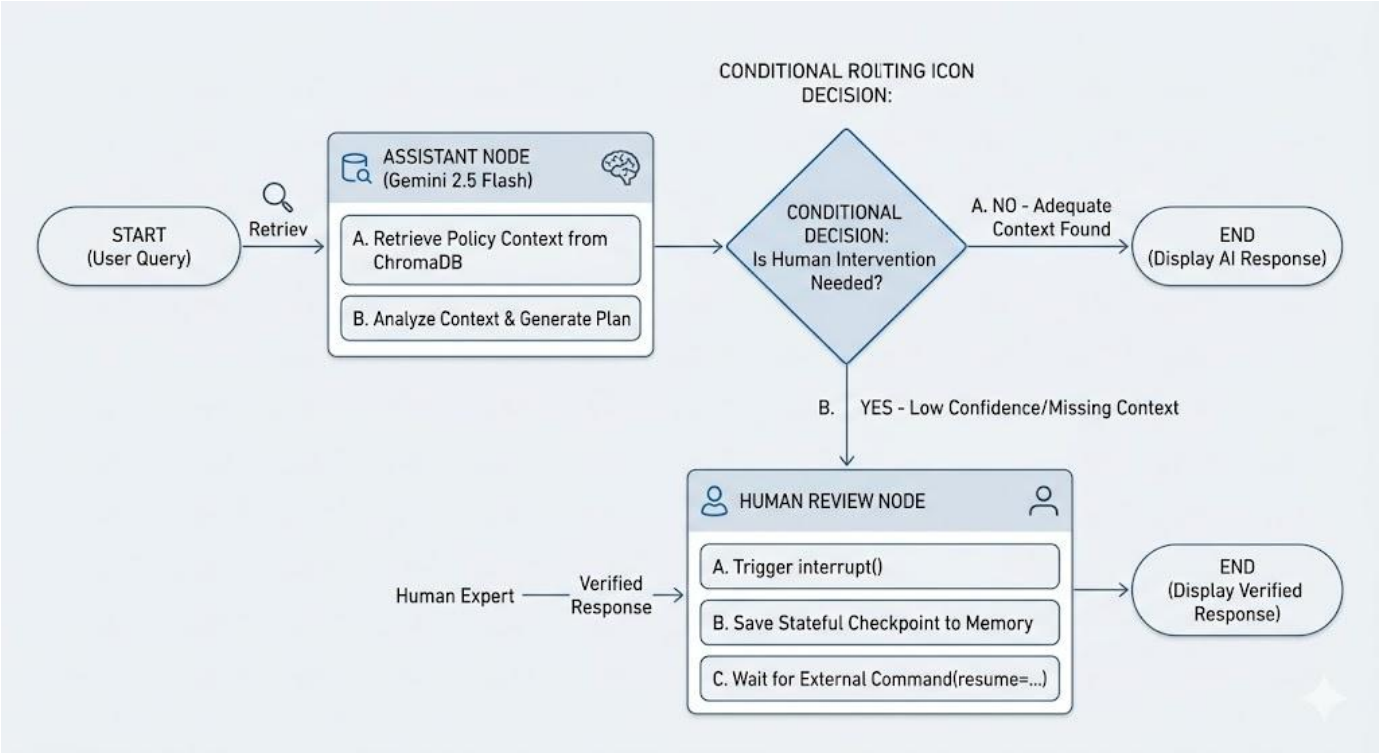

Figure 1: LangGraph Stateful Execution Flow showing automated retrieval and human-in-the-loop transition.

4. Conditional Routing Logic

The "intelligence" of the workflow lies in the conditional gateway between the Assistant and the Human Reviewer. This logic prevents the model from guessing when it lacks information.

4.1 Routing Decision Gate

Following the **Assistant Node**, the graph evaluates the output using a specific routing function. The decision is based on a binary flag, `needs_escalation`, which is determined by the following criteria:

- **Path A: NO (Direct Response):** Triggered when Gemini 2.5 Flash Lite finds a high-confidence match within the University of Arkansas Extension PDF. The graph routes directly to **END**.
- **Path B: YES (Escalation):** Triggered under the following conditions:
  - ✓ **Context Gap:** The retrieved chunks do not contain the answer to the user's specific question.
  - ✓ **Ambiguity:** The query requires a policy interpretation that falls outside the scope of the provided document.
  - ✓ **Explicit Token:** The LLM outputs the predefined `ESCALATE` token.

4.2 Impact on System Reliability

By implementing this conditional routing, we ensure a "Zero-Hallucination" environment. Instead of providing a potentially incorrect answer, the system gracefully redirects the query to the **Human Review Node**, as illustrated in the workflow diagram.

5. HITL Design & Integration

This module manages the user-human-AI interface.

```python
from langgraph.types import Command

# 3. Check if the graph is waiting for a human
snapshot = app.get_state(config)

if snapshot.next:
    print(f"\n[!] SYSTEM PAUSED at node: {snapshot.next}")
    print("The AI has requested human intervention.")

    # 4. Provide the manual expert response
    # You can look up the answer in the Arkansas PDF (e.g., 'Return calls within 24 hours')
    human_expert_answer = input("Enter the expert response to provide to the user: ")

    # 5. RESUME the graph using Command(resume=...)
    print("\n--- RESUMING WITH EXPERT INPUT ---")
    for event in app.stream(Command(resume=human_expert_answer), config, stream_mode="values"):
        if "messages" in event:
            print(f"Final Verified Response: {event['messages'][-1].content}")
else:
    print("\n[√] The Assistant answered successfully using the PDF context.")
```

```
[!] SYSTEM PAUSED at node: ('human_review',)
The AI has requested human intervention.
Enter the expert response to provide to the user:
```

- **When escalation is triggered:** The graph hits a breakpoint and saves the current state to the `MemorySaver` checkpointer.
- **What happens after escalation:** The system prompts the user/expert in the console to provide a "Verified Response."
- **How human response is integrated:** The expert's input is passed via `Command(resume=...)`, replacing the AI's "ESCALATE" token with the expert's answer.

6. API / Interface Design

- **Input format:** String (User question).
- **Output format:** String (Assistant response or Expert response).
- **Interaction flow:** Synchronous stream from the user → Asynchronous wait for the human → Final output display.

7. Error Handling

- **Missing data:** Handled by the `retriever` returning an empty list, which triggers immediate escalation.
- **No relevant chunks found:** The LLM is prompted to recognize "No Information Found" and output the `ESCALATE` signal.
- **LLM failure:** Implemented via a `try-except` block to catch API errors (like the 503 error) and retry the request.